Projeto 2: Classificação Vetorial de Entidades

Pipeline NLP: N-grams, TF-IDF e Grafos Clínicos

Luiz Castro – 150731 Rafael Setton – 214381
Victor Itiro – 244075 Yan Oliveira – 236363

Machine Learning em Texto Clínico

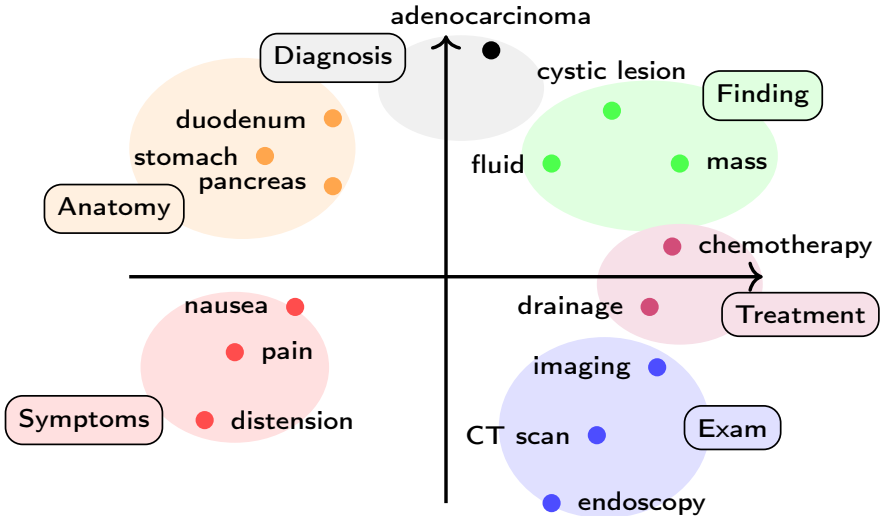
Repositório: github.com/Cl4nyz/Natural-Language-Processing

November 9, 2026

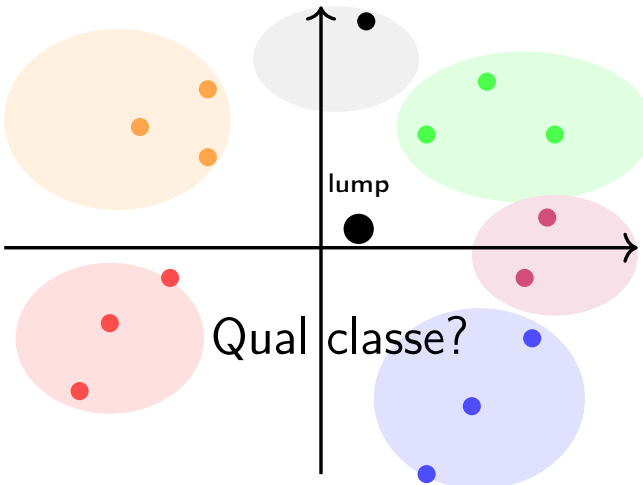
Visão geral: Projeto 1 vs Projeto 2

- **O Problema (Projeto 1):** Dependência de um dicionário rígido de busca exata. Expressões clínicas complexas caíam em `Unknown`.
- **A Solução (Projeto 2):** Converter textos em vetores (TF-IDF + N-grams) e comparar a semelhança angular.
- **Aplicação Prática:** O código Python processa texto, extrai sintagmas nominais e os posiciona em um espaço vetorial de características.
- **Objetivo Final:** Extrair entidades clínicas flexíveis e conectá-las em um Grafo de Conhecimento semântico usando `NetworkX`.

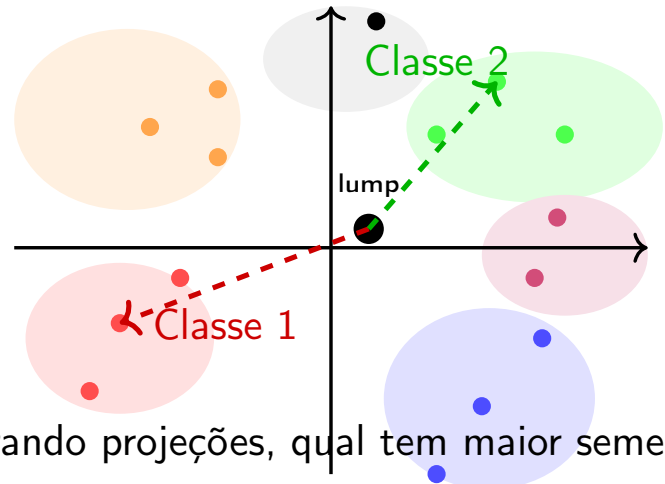
Ideia central: Espaço vetorial das categorias



Ideia central: Espaço vetorial das categorias

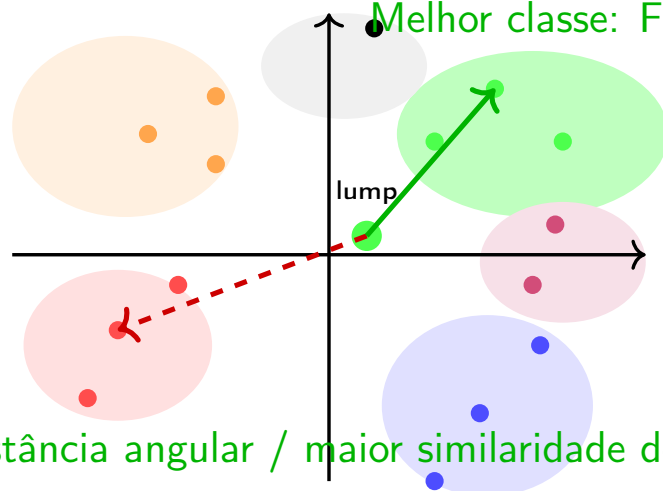


Ideia central: Espaço vetorial das categorias



Comparando projeções, qual tem maior semelhança?

Ideia central: Espaço vetorial das categorias



Menor distância angular / maior similaridade de cosseno

Algoritmo Matemático: TF-IDF

1. Frequência do Termo (TF):

$$\mathrm{TF}(t, d) = f_{t,d}$$

Mede a frequência do termo t no documento d (contagem direta no `scikit-learn`).

2. Frequência Inversa nos Documentos (IDF):

$$\mathrm{IDF}(t, D) = \ln\left(\frac{1 + N}{1 + \mathrm{df}(t)}\right) + 1$$

Mede quão raro é o termo no corpus D, onde N é o total de documentos e $\mathrm{df}(t)$ é o número de documentos contendo t (suavização padrão *smooth_idf=True*).

3. Vetor TF-IDF e Normalização L_2:

$$\text{TF-IDF}(t, d, D) = \mathrm{TF}(t, d) \times \mathrm{IDF}(t, D), \qquad \hat{v} = \frac{v}{\|v\|_2}$$

O vetor resultante é normalizado pela norma euclidiana $\|v\|_2 = \sqrt{\sum v_i^2}$ para equilibrar documentos de tamanhos distintos.

No Código: O Espaço de N-Grams e TF-IDF

Inicializando o Espaço:

- Mapeamos os termos do dicionário para um espaço **contínuo**.
- O `TfidfVectorizer` do scikit-learn gera pesos baseados em frequência local (TF) e global (IDF).
- Unigramas e bigramas (*ngram_range=(1,2)*) capturam conexões de palavras (ex: *cystic lesion*).

No Código: O Espaço de N-Grams e TF-IDF

Inicializando o Espaço:

- Mapeamos os termos do dicionário para um espaço **contínuo**.
- O `TfidfVectorizer` do scikit-learn gera pesos baseados em frequência local (TF) e global (IDF).
- Unigramas e bigramas (*ngram_range=(1,2)*) capturam conexões de palavras (ex: *cystic lesion*).

```
def build_ngram_vector_space():
    # ... carrega vocabulario ...
    # Cria o espaco vetorial com TF-IDF de unigramas e bigramas
    vectorizer = TfidfVectorizer(ngram_range=(1, 2), analyzer='word')
    X_train_vectors = vectorizer.fit_transform(train_terms)

    return vectorizer, X_train_vectors, train_categories
```

Algoritmo Matemático: Similaridade de Cosseno

Definição Geral da Similaridade de Cosseno:

$$\text{sim_cosseno}(u, v) = \frac{u \cdot v}{\|u\|\|v\|}$$

Mede o cosseno do ângulo entre dois vetores no espaço (1 para vetores com a mesma direção, 0 para ortogonais).

Regra de Decisão com Limiar (*Threshold*):

$$\hat{c} = \begin{cases} \text{Categoria}(\arg\max_j \text{sim}(u, v_j)), & \text{se } \max_j \text{sim}(u, v_j) \geq \tau \\ \text{"Unknown"}, & \text{caso contrário} \end{cases}$$

Onde $\tau = 0.35$ é o limiar de decisão estipulado no código.

No Código: Similaridade de Cosseno

Tomada de Decisão:

- Novo termo encontrado é projetado no mesmo espaço usando `transform()`.
- Calculamos a distância de cosseno (*Nearest Neighbor*) entre o termo novo e todos os protótipos conhecidos.
- Definimos um limiar (*threshold* ≥ 0.35): se a similaridade for baixa, classifica como `Unknown`.

No Código: Similaridade de Cosseno

Tomada de Decisão:

- Novo termo encontrado é projetado no mesmo espaço usando `transform()`.
- Calculamos a distância de cosseno (*Nearest Neighbor*) entre o termo novo e todos os protótipos conhecidos.
- Definimos um limiar (*threshold* ≥ 0.35): se a similaridade for baixa, classifica como `Unknown`.

```
def classify_entity_vector_space(entity_str, threshold=0.35):
    # Projeta no espaco e compara com cosine_similarity
    entity_vector = VECTORIZER.transform([entity_lower])
    cosine_sims = cosine_similarity(entity_vector, X_VOCAB)[0]

    best_idx = cosine_sims.argmax()
    if cosine_sims[best_idx] >= threshold:
        return TRAIN_CATEGORIES[best_idx]

    return "Unknown"
```

No Código: Extração com NLTK

- Como achar as entidades antes de classificar? Usamos POS-Tagging e *Regex Parsing*.
- Buscamos sintagmas nominais (NP) que possam representar termos médicos.

```
def extract_noun_chunks(tokens):
    tagged_tokens = pos_tag(tokens)
    # Define a gramatica do Chunk (Adjetivos + Substantivos)
    grammar = r"NP: {<DT>?<JJ.*|CD|NN.*>+(?:\s+<CC>\s+<JJ.*|CD|NN.*>+)*}"
    chunk_parser = RegexpParser(grammar)
    tree = chunk_parser.parse(tagged_tokens)

    # Extrai, limpa os 'stops words' e envia para a classificacao
    # ...
```

Construção do Grafo de Conhecimento

Conectando os nós com NetworkX:

- Após extrair as entidades (*nodes*), precisamos definir as arestas (*edges*).
- O código infere relações pelo contexto da frase (ex: presença de verbos como *"revealed"*, *"showed"* ou preposições espaciais).

Construção do Grafo de Conhecimento

Conectando os nós com NetworkX:

- Após extrair as entidades (*nodes*), precisamos definir as arestas (*edges*).
- O código infere relações pelo contexto da frase (ex: presença de verbos como *"revealed"*, *"showed"* ou preposições espaciais).

```
def build_generic_graph(text):
    G = nx.DiGraph()
    # ... Extrai entidades ...

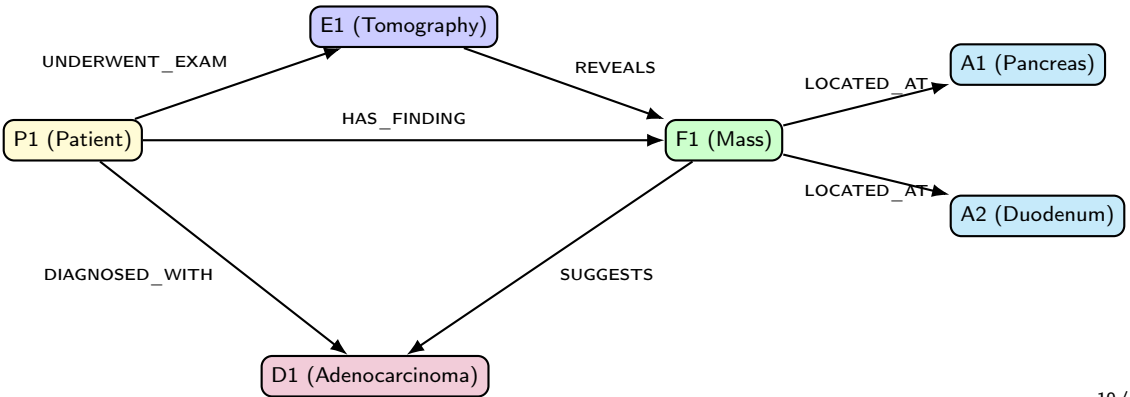
    if entity_type == "Finding":
        if last_exam and any(verb in sentence for verb in VERBS_REVEAL):
            G.add_edge(last_exam, entity_text, label="REVEALS")

    elif entity_type == "Anatomy":
        if last_finding and any(prep in sentence for prep in PREP_SPATIAL):
            G.add_edge(last_finding, entity_text, label="LOCATED_AT")
```

Exemplo Prático do Grafo

Entidades Multiconectadas

No Projeto 2, o grafo evidencia múltiplos relacionamentos. O nó **Central mesenteric mass** (F1) atua como um hub central ligando exames, anatomia e diagnósticos.



Resultados e Comparação

- **Generalização no Projeto 2:** Diferente do Projeto 1, o sistema atual reconhece termos não exatos calculando a semelhança no espaço vetorial gerado pelo *TF-IDF*.
- **Robustez a Ruído:** Frases cotidianas sem similaridade ≥ 0.35 caem automaticamente em `Unknown`, removendo os falsos-positivos sem necessidade de regras condicionais complexas.
- **Interpretação e Visualização:** O modelo produz um *Knowledge Graph* (Grafo de Conhecimento) exportável para *Mermaid*, permitindo visualizar claramente o "raciocínio" sintático e semântico construído do texto.

Obrigado!

Dúvidas?